

Universidad de Buenos Aires
Facultad de Ciencias Exactas y Naturales
Departamento de Computación

Typestate Checker: Análisis de una herramienta de verificación de Object Protocols

Tesis de Licenciatura en Ciencias de la Computación

Manuel Ignacio Lakowsky

Director: Sebastián Uchitel

Buenos Aires, 2026

TYPESTATE CHECKER: ANÁLISIS DE UNA UNA HERRAMIENTA DE VERIFICACIÓN DE OBJECT PROTOCOLS

Diversas herramientas y técnicas se han desarrollado a lo largo del tiempo para detectar potenciales errores en programas. En particular, los sistemas de tipado ...

Palabras claves: Guerra, Rebelión, Wookie, Jedi, Fuerza, Imperio (no menos de 5).

TYPESTATE CHECKER: AN ANALYSIS OF AN OBJECT PROTOCOLS CHECKER TOOL

In a galaxy far, far away, a psychopathic emperor and his most trusted servant – a former Jedi Knight known as Darth Vader – are ruling a universe with fear. They have built a horrifying weapon known as the Death Star, a giant battle station capable of annihilating a world in less than a second. When the Death Star's master plans are captured by the fledgling Rebel Alliance, Vader starts a pursuit of the ship carrying them. A young dissident Senator, Leia Organa, is aboard the ship & puts the plans into a maintenance robot named R2-D2. Although she is captured, the Death Star plans cannot be found, as R2 & his companion, a tall robot named C-3PO, have escaped to the desert world of Tatooine below. Through a series of mishaps, the robots end up in the hands of a farm boy named Luke Skywalker, who lives with his Uncle Owen & Aunt Beru. Owen & Beru are viciously murdered by the Empire's stormtroopers who are trying to recover the plans, and Luke & the robots meet with former Jedi Knight Obi-Wan Kenobi to try to return the plans to Leia Organa's home, Alderaan. After contracting a pilot named Han Solo & his Wookiee companion Chewbacca, they escape an Imperial blockade. But when they reach Alderaan's coordinates, they find it destroyed - by the Death Star. They soon find themselves caught in a tractor beam & pulled into the Death Star. Although they rescue Leia Organa from the Death Star after a series of narrow escapes, Kenobi becomes one with the Force after being killed by his former pupil - Darth Vader. They reach the Alliance's base on Yavin's fourth moon, but the Imperials are in hot pursuit with the Death Star, and plan to annihilate the Rebel base. The Rebels must quickly find a way to eliminate the Death Star before it destroys them as it did Alderaan (aprox. 200 palabras).

Keywords: War, Rebellion, Wookie, Jedi, The Force, Empire (no menos de 5).

AGRADECIMIENTOS

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Fusce sapien ipsum, aliquet eget convallis at, adipiscing non odio. Donec porttitor tincidunt cursus. In tellus dui, varius sed scelerisque faucibus, sagittis non magna. Vestibulum ante ipsum primis in faucibus orci luctus et ultrices posuere cubilia Curae; Mauris et luctus justo. Class aptent taciti sociosqu ad litora torquent per conubia nostra, per inceptos himenaeos. Mauris sit amet purus massa, sed sodales justo. Mauris id mi sed orci porttitor dictum. Donec vitae mi non leo consectetur tempus vel et sapien. Curabitur enim quam, sollicitudin id iaculis id, congue euismod diam. Sed in eros nec urna lacinia porttitor ut vitae nulla. Ut mattis, erat et laoreet feugiat, lacus urna hendrerit nisi, at tincidunt dui justo at felis. Class aptent taciti sociosqu ad litora torquent per conubia nostra, per inceptos himenaeos. Ut iaculis euismod magna et consequat. Mauris eu augue in ipsum elementum dictum. Sed accumsan, velit vel vehicula dignissim, nibh tellus consequat metus, vel fringilla neque dolor in dolor. Aliquam ac justo ut lectus iaculis pharetra vitae sed turpis. Aliquam pulvinar lorem vel ipsum auctor et hendrerit nisl molestie. Donec id felis nec ante placerat vehicula. Sed lacus risus, aliquet vel facilisis eu, placerat vitae augue.

A mi persona favorita.

Índice general

1.. Introducción 1
1.1. Estructura de la Tesis 1

2.. Trabajos Relacionados 3
2.1. Typestates y Object Protocols 3
2.1.1. Inicios 3
2.1.2. Programación Orientada a Objetos 3
2.1.3. Uso de Protocolos 4
2.2. Plaid 4
2.3. Mungo 5
2.3.1. Sintaxis y funcionalidad 6
2.3.2. Implementación 6
2.3.3. Estado del proyecto 7

3.. Java Typestate Checker 8
3.1. Definición de Protocolos 8
3.1.1. Estados Descartables 9
3.1.2. Múltiples Destinos 9
3.2. Aliasing 10
3.3. Anotaciones Adicionales 11
3.3.1. Nullable 11
3.3.2. Requires 11
3.3.3. Ensures 12
3.4. Subtipado 13
3.5. Impacto 15
3.6. Implementación 15
3.6.1. Algoritmo 15
3.6.2. Checker Framework 15

1. INTRODUCCIÓN

A lo largo de los años, diversas herramientas se han desarrollado para detectar lo antes posible errores en programas. Desde sistemas de tipado y análisis estático de código, hasta frameworks de testing unitario y/o automático, se busca agregar robustez al software y permitirle al desarrollador tener más confianza sobre lo que produce.

Estos potenciales errores en programas pueden ser agrupados en diferentes categorías. Por ejemplo, los errores aritméticos, que ocurren al realizar operaciones matemáticas indebidas, tales como dividir por cero, o errores de punteros, que suceden al desreferenciar punteros nulos. Un grupo particular de errores son los relacionados a los **Object Protocols** en lenguajes de programación orientados a objetos. En pocas palabras, los protocolos de los objetos son los que determinan las secuencias de métodos válidas para los mismos. Uno de los ejemplos típicos de un protocolo se encuentra en los lectores de archivos, donde antes de leer hay que abrir el descriptor del archivo, y al finalizar hay que cerrarlo. Otro ejemplo es el de los `Iterator`s en Java[18], donde el método `remove()` solamente puede ser llamado una única vez luego de cada llamado a `next()`. Pensar en object protocols no es inusual, y múltiples instancias de uso y definición de estos protocolos pueden encontrarse en varios proyectos open source[4].

De no cumplir con el protocolo de un objeto al usar sus métodos pueden generarse excepciones inesperadas, y en algunos casos el comportamiento de los mismos puede ser indefinido. Es por esto que cobran importancia las herramientas de detección de incumplimiento de object protocols, las cuales, analizando el código estática o dinámicamente, permiten avisarle lo antes posible al desarrollador de un potencial error.

En este trabajo enfocaremos nuestro estudio en una de estas herramientas llamada **Java Typestate Checker**[16]. *JaTyC* analiza código de Java estáticamente para determinar si el uso de un objeto por parte de algún cliente rompe el protocolo del mismo. Estos protocolos son definidos por los mismos programadores a través de **Typestates**, los diferentes estados por los que puede pasar el objeto y los métodos habilitados en cada uno.

Está basada en **Mungo**[21], un conjunto de herramientas de investigación universitarias centradas en la verificación de protocolos, agregando mejoras y funcionalidades innovadoras[15]. A su vez, está escrita como plugin y mencionada en la documentación oficial del **Checker Framework**[8], en el que confían empresas tales como Amazon[2][3] y que ha sido utilizado en incontables publicaciones. En ese sentido, *JaTyC* se posiciona como uno de los analizadores más flexibles y contemporáneos para object protocols en Java.

A lo largo de esta investigación realizaremos múltiples pruebas con la herramienta, aprendiendo sobre su poder de expresión, las garantías que provee y sus limitaciones. Buscamos mejorar el entendimiento del alcance de *JaTyC*, y a su vez de la verificación de object protocols a través de typestates en general.

1.1. Estructura de la Tesis

En el capítulo 2 se dará un poco de contexto teórico y práctico alrededor de los object protocols y typestates. Se mencionarán publicaciones relacionadas y herramien-

tas/lenguajes predecesoras a *JaTyC*.

El capítulo 3 se centrará en la herramienta, explicando qué es y cómo se usa. Se mencionará además cómo funciona internamente, cuál es su algoritmo de análisis estático de código, y cómo se apoya en el *Checker Framework*.

El capítulo 4 recopilará los hallazgos de la experimentación sobre *JaTyC*. Se dividirá en distintas secciones según los aspectos o funcionalidades de la herramienta a poner bajo la lupa.

En el capítulo 5 se hará un resumen de lo aprendido, se establecerán conclusiones, y se discutirá trabajo futuro.

2. TRABAJOS RELACIONADOS

2.1. Typestates y Object Protocols

2.1.1. Inicios

La noción de **typestate** fue introducida en 1986 por Strom y Yemini [19]. A pesar de que ya en esa época existían mecanismos de prevención de errores en programas, tales como los sistemas de tipado o reglas basadas en scopes estáticos, los autores identificaron una limitación en los mismos. Específicamente, no estaban diseñados para detectar ciertas operaciones prohibidas en algunos contextos, tales como desreferenciar un puntero apuntando a memoria ya liberada, o utilizar una variable antes de ser inicializada. Para cubrir esa necesidad definieron los **typestates**, los cuales intentan modelar el estado en el que se encuentra un objeto a la hora de realizar una operación. Mientras que los tipos restringen las operaciones permitidas para un dato/objeto, y los scopes el acceso a variables fuera de su ciclo de vida, los typestates intentan restringir el orden en el cual las operaciones suceden bajo un contexto particular.

El funcionamiento de los typestates es el siguiente. Dado un tipo, se le asocia un conjunto de typestates, y se define para cada uno un conjunto de operaciones permitidas. A su vez, se definen reglas de transición entre ellos a través de estas operaciones. En cualquier punto del programa, un objeto estará en alguno de estos typestates y podrán ser aplicadas solamente sus operaciones permitidas. Al interactuar con él, podrá transicionar a otro typestate según las reglas definidas, cambiando el conjunto de operaciones disponibles.

El paper menciona además conceptos que herramientas contemporáneas como *JaTyC* implementan, tales como el de **completitud de protocolo**, donde se espera que los objetos lleguen a un estado final antes de que el programa termine. Según los autores, una ejecución es *typestate-correcta* si y solo si todas las operaciones realizadas en objetos son permitidas por el typestate que las acompaña, y todos los objetos completan su protocolo.

Finalmente, el trabajo establece las definiciones formales alrededor de los typestates para trabajar con ellos. Con las mismas, presenta un algoritmo para analizar estáticamente el cumplimiento de protocolos en programas.

2.1.2. Programación Orientada a Objetos

Estas ideas fueron posteriormente extendidas por Bierhoff y Aldrich [6], llevándolas hacia el mundo de la programación orientada a objetos.

Por un lado, introducen y formalizan la noción de *refinamiento de estados*, utilizada cuando se quiere hacer herencia de un tipo ya protocolado. Si el subtipo quiere agregar nuevos typestates a su protocolo, debe asegurarse necesariamente que se sigan cumpliendo las restricciones del protocolo del supertipo que extiende. En otras palabras, toda secuencia de llamadas de métodos válidas en el tipo padre debe mantenerse en el protocolo del tipo hijo. El refinamiento de estados habla sobre cómo un subconjunto de estados en el subtipo son "subcasos" de un solo estado en el supertipo, los cuales pueden agregar nuevas secuencias de métodos válidas al protocolo, pero no quitar las preexistentes. En cierto sentido, si modelamos el protocolo de un objeto como una máquina de estados finita, el autómata del subtipo debe preservar el comportamiento del autómata del supertipo.

Por otro lado, introducen la idea de *dimensiones de estados*. Esta noción busca dar respuesta a situaciones en las que conceptualmente se considera que un objeto está en varios typestates a la vez, atados a variables completamente independientes. Estos subconjuntos de estados son referidos en el trabajo como *ortogonales*. Por ejemplo, observemos la interfaz de la figura 2.1:

```
1 public interface Iterator {
2     public boolean hasNext();
3     public int next();
4     public boolean canPrint();
5     public void print();
6 }
```

Fig. 2.1: Ejemplo de tipo con typestates ortogonales

Puede observarse que el comportamiento de la interfaz depende de si se puede seguir iterando sobre la colección y de si actualmente se puede imprimir. Asumiendo que ambas condiciones son independientes entre sí, definir un protocolo que correctamente especifique las cadenas de métodos válidas puede resultar tedioso y conceptualmente incorrecto. Esto es porque, para modelar el protocolo lo más detallado posible, se necesitaría un typestate por cada elemento del producto cartesiano entre los conjuntos $\{PuedoImprimir, NoPuedoImprimir\}$ y $\{TengoSiguiente, NoTengoSiguiente\}$. Puede verse fácilmente que, con un tipo que tenga un gran número de variables ortogonales de las cuales dependen los typestates, la definición del protocolo puede dispararse en tamaño. A su vez, puede que tener un estado como *TengoSiguienteImprimible* no tenga mucho sentido.

Para solventar esto, el paper define que el objeto está en varias *dimensiones de estados* ortogonales a la vez, las cuales permiten distintas secuencias de llamadas de métodos. Para nuestro ejemplo, la interfaz puede encontrarse en el estado (*PuedoImprimir*, *TengoSiguiente*), permitiéndole acceso a los métodos `print()` y `next()`. Esto evita la explosión de cantidad de estados, mejora la legibilidad y claridad del protocolo, y también permite acercarnos a utilizar typestates para herencia múltiple (definiendo los protocolos heredados como ortogonales).

2.1.3. Uso de Protocolos

Finalmente, en el trabajo de Beckman et al. [5] se realizó un estudio sobre el uso de *object protocols* en proyectos open source y la librería estándar de Java. En él se intenta describir qué tanto se suelen definir protocolos, cuánto se utilizan y se respetan por código cliente, y qué estructura suelen tener. Se descubrió que, a lo largo del código analizado, aproximadamente 7 % de todos los tipos definían protocolos, y más del 10 % de las clases eran clientes de tipos protocolados. Dado que la muestra de código analizada fue significativa, esto nos indica que las herramientas de verificación de protocolos tales como *JaTyC* son potencialmente aplicables a numerosos proyectos.

2.2. Plaid

A raíz del surgimiento del concepto de typestates, se ideó un nuevo paradigma de programación: la Programación Orientada a Typestates (TSOP)[9]. Bajo este paradigma,

el código no solo es modelado alrededor de clases sino también sus typestates y protocolos. En otras palabras, pasan a ser *first-class citizens* (entidades de primer orden).

Plaid[20][1] es uno de los primeros lenguajes de programación orientada a typestates, desarrollado por la Universidad Carnegie Mellon. En Plaid, los objetos siempre se encuentran en un typestate particular, los cuales se definen en forma similar a las clases tradicionales (con métodos, variables, subestados, etc). Estos pueden transicionar durante la ejecución del programa entre un typestate y otro, modificando su interfaz y comportamiento.

Un ejemplo puede observarse en la Figura 2.2. En ella, se define la interfaz `File` a través de los dos typestates `ClosedFile` y `OpenFile`. Mientras que la variable `filename` está disponible en todos los estados, hay métodos exclusivos para cuando los objetos de tipo `File` estén abiertos o cerrados. A su vez, se definen en los métodos las transiciones entre typestates, como por ejemplo en `[ClosedFile≫OpenFile]`, transicionando un objeto en el estado cerrado al abierto. Esto puede hacerse tanto para el objeto receptor del método como para sus argumentos.

```
1  state File {
2      val String filename;
3  }
4
5  state ClosedFile = File with {
6      method void open() [ClosedFile≫OpenFile];
7  }
8  state OpenFile = File with {
9      private val CFile fileResource;
10
11     method int read();
12     method void close() [OpenFile≫ClosedFile];
13 }
```

Fig. 2.2: Ejemplo de definición de clase y typestates con Plaid

Entre otras funcionalidades, Plaid cuenta con un sistema de permisos de acceso[7] para intentar solucionar el problema de la verificación de protocolos en la presencia de *aliasing*. Finalmente, similar a *JaTyC*, Plaid cuenta con un analizador estático que permite verificar el uso correcto de los typestates y los protocolos de los objetos en tiempo de compilación.

2.3. Mungo

Mungo[21][13] es una herramienta predecesora de *JaTyC* para Java que, similarmente, verifica estáticamente que se sigan los protocolos de objetos. Esto lo consigue a través de la abstracción de typestates, modelando los diferentes estados por los que pasa una clase, y reporta errores cuando código cliente no respeta el protocolo de las mismas. En la documentación oficial de *JaTyC*[17][14], los autores hacen hincapié en que Mungo es la herramienta principal en la que se basaron para su desarrollo, incluso mencionando que el plan inicial era simplemente extenderla con nuevas funcionalidades. En cambio, *JaTyC* terminó siendo una herramienta independiente, y es de interés notar las diferencias entre ambas[15].

2.3.1. Sintaxis y funcionalidad

Comenzando por la funcionalidad y sintaxis, las dos herramientas son bastante similares. Las definiciones de los protocolos y sus typestates son escritas en archivos separados con extensión *.protocol* (ver Figura 2.3). Luego, para referenciar un protocolo desde una clase se la anota con "`@Typestate(<protocolo>)`". Cuando el código fuente de Java es compilado, primero es analizado estáticamente por el mismo sistema de tipado del lenguaje, y luego por el de Mungo. Si una clase está asociada a un protocolo, sus clientes deben respetar el correcto orden de llamados de sus métodos. De lo contrario, la herramienta reporta un error.

```
typestate StackProtocol {
    Empty = {
        void push(int): NonEmpty,
        void deallocate(): end
    }
    NonEmpty = {
        void push(int): NonEmpty,
        int pop(): Unknown
    }
    Unknown = {
        void push(int): NonEmpty,
        Check isEmpty(): <EMPTY: Empty, NONEMPTY: NonEmpty>
    }
}
```

Fig. 2.3: Ejemplo de definición de protocolo para un Stack en Mungo

Los protocolos se escriben con una sintaxis muy similar a la de *JaTyC*, e incluso idéntica en muchos casos. En la Figura 2.3 puede observarse un ejemplo oficial de cómo se vería un protocolo para una clase `Stack`, que permite apilar infinitamente y solo permite desapilar si no está vacío. Cuenta con tres typestates, uno para cuando la pila está vacía `Empty`, uno para cuando la pila tiene algún elemento `NonEmpty`, y otro para cuando no se sabe el estado actual de la misma `Unknown`. Cada typestate describe la signatura de los métodos permitidos en cada uno, y el (o los) typestate al que pasaría el objeto de llamarlos. Al ser instanciada, la pila comienza siempre en el primer typestate definido en el protocolo, en este caso `Empty`, y finaliza su protocolo al transicionar al typestate reservado `end` llamando al método `deallocate()`.

Un aspecto en el que *JaTyC* mejora con respecto a su antecesor es la funcionalidad. Información detallada puede encontrarse en el cuadro comparativo oficial de la herramienta[15], pero las principales utilidades que agrega son los estados *droppable*, el subtipado, y verificación de typestates en argumentos y valores de retorno de métodos. A su vez, los autores recalcan que Mungo falla al manejar ciertos casos borde, relacionados con valores nulos, linealidad de las variables y corroborar la finalización de los protocolos. Profundizaremos más en todos estos conceptos en el próximo capítulo.

2.3.2. Implementación

Con respecto a la implementación de ambas herramientas, Mungo está creada sobre el framework **JastAdd**[10]. Este es un sistema de meta-compilación fácilmente extensible, que le permite a sus usuarios crear herramientas de análisis estático customizables. A pesar

de haber sido utilizado para múltiples proyectos[11], JastAdd no se mantiene activamente, con solamente 3 releases desde 2020.

En comparación, *JaTyC* está implementada en el **Checker Framework**[8], que extiende el sistema de tipado de Java. El framework le permite a los desarrolladores incluir numerosos plug-ins, o checkers, a sus proyectos, los cuales realizan diferentes tipos de verificaciones en tiempo de compilación. Uno de los checkers más conocidos del framework es el *Nullness Checker*, el cual verifica que no pueda suceder ninguna excepción de `null pointers` durante ejecuciones. Cuenta con varios otros plug-ins útiles, tales como el *SQL Quotes Checker* o el *Regex Checker*. Es un framework mantenido activamente, y es utilizado en la industria, por ejemplo por Amazon con sus checkers propios *KMS Compliance Checker*[3] y *Crypto Policy Compliance Checker*[2].

Por otro lado, Mungo está desarrollada en Java. Parte de la razón por la que los autores de *JaTyC* decidieron hacerla una herramienta independiente, y no una extensión de Mungo, fue poder escribirla en Kotlin[12]. Ellos destacan que la interoperabilidad entre ambos lenguajes (que permite reutilizar librerías de Java), junto con la sintaxis más concisa de Kotlin y sus funcionalidades de seguridad, hicieron más productivo el desarrollo.

2.3.3. Estado del proyecto

Finalmente, el repositorio de Mungo[22] muestra que el último commit se realizó en 2020, por lo que podemos asumir que el desarrollo de la herramienta está inactivo. Por otro lado, *JaTyC* estuvo siendo actualizada hasta recientemente, con su último commit subido en 2025.

3. JAVA TYPESTATE CHECKER

Java Typestate Checker[16] es una herramienta de verificación de código Java para protocolos de objetos. Analizando el programa en forma estática, detecta errores relacionados con el orden de las llamadas a métodos, consumo debido de los recursos y herencia de protocolos, entre otras cosas. La herramienta está escrita en Kotlin[12] como plug-in del Checker Framework[8], y se destaca por ser una de las herramientas de análisis de object protocols más flexibles y actualizadas para Java. Son estas las razones por las cuales seleccionamos esta herramienta para analizar el poder de expresión y limitaciones de la verificación de object protocols.

3.1. Definición de Protocolos

JaTyC permite a sus usuarios definir protocolos para sus clases a través de **typestates**. Dentro de cada uno de ellos, se establece la interfaz para los objetos que estén en ese estado, definiendo los métodos disponibles y las transiciones resultantes de llamarlos.

```
typestate Iterator {
  HasNext = {
    boolean hasNext(): <true: Next, false: end>
  }
  Next = {
    boolean hasNext(): <true: Next, false: end>,
    Object next(): HasNext
  }
}
```

Fig. 3.1: Definición de protocolo simple para un Iterador

```
import jatyc.lib.Typestate;

@Typestate("Iterator")
public interface Iterator {
  public boolean hasNext();
  public int next();
}
```

Fig. 3.2: Suscripción de protocolo a través de la anotación `@Typestate`

Un ejemplo simple de un protocolo puede observarse en la Figura 3.1. En el mismo, se definen dos typestates para una clase `Iterator`: `HasNext` y `Next`. Al ser el primer estado definido, `HasNext` es el inicial para cualquier instancia de la clase, y cuenta solamente con el método `hasNext()` disponible. Al ser llamado, el objeto puede transicionar a diferentes estados según el resultado booleano retornado: al estado `Next` de ser verdadero o al estado reservado `end` de ser falso, que señala la terminación del protocolo. De finalizar el protocolo, no se podrán llamar más ninguno de sus métodos (con la excepción de los métodos *anytime*, de los cuales hablaremos en la seccion 3.2). De transicionar a `Next`, el iterador podrá llamar

a ambos métodos `hasNext()` y `next()`, donde el último devuelve el siguiente elemento en la colección y regresa a la instancia al estado inicial. En resumen, el protocolo indica que el objeto solamente puede pedir el siguiente elemento de estar seguro de que exista, y de no haber más elementos termina su ciclo de vida. Los clientes del mismo deberán de respetar este comportamiento, y de no hacerlo *JaTyC* alertará un error.

Anotando una clase con `@Typestate(<protocolo>)` se la suscribe al protocolo, como muestra la figura 3.2.

3.1.1. Estados Descartables

Los **Droppable States**, o "estados descartables", son parte de la funcionalidad nueva introducida por la herramienta comparada con Mungo. Los mismos proveen una alternativa para el programador a la hora de indicar la finalización del protocolo de un objeto.

```
typestate Animal {
  Init = {
    void move(): Moved,
    drop: end
  }
  Moved = {
    void eat(): Init
  }
}
```

Fig. 3.3: Protocolo con estados descartables

En la Figura 3.3 puede observarse cómo funcionan. El estado inicial `Init` tiene declarado un único método llamable, `move()`, y a su vez que es descartable, con la notación `drop: end`. La misma indica que a la hora de finalizar el ciclo de vida del objeto, de encontrarse en el estado `Init`, transicionará automáticamente a `end`, finalizando su protocolo.

Lo que permiten los estados descartables es más flexibilidad a la hora de definir protocolos, sobre todo cuando no hay un método claro que debería de transicionar al objeto a `end`. Cualquier estado puede ser descartable, y varios estados pueden ser descartables a la vez.

3.1.2. Múltiples Destinos

A la hora de definir un protocolo, hay situaciones en las que podría ser deseable expresar, para un mismo método, transiciones a typestates diferentes según el resultado de la llamada.

JaTyC permite definir múltiples destinos para resultados booleanos o enumeraciones. Por ejemplo, en la Figura 3.1, podemos observar cómo para el método `hasNext()` se define que el objeto pasará al estado `Next` o `end` según si el resultado booleano devuelve verdadero o falso. Por otro lado, en la Figura 3.4 se aprecia un protocolo para un lector de archivos. En él, el estado inicial `Init` define un método `open()` el cual devuelve una enumeración de tipo `FileStatus`, con potenciales valores `OK` y `ERROR`. Según el resultado devuelto, el objeto pasará a estar habilitado a leer el archivo abierto o finalizará su protocolo.

```
typestate FileProtocol {
  Init = {
    FileStatus open(): <OK: Read, ERROR: end>
  }
  Read = {
    String read(): Close
  }
  Close = {
    void close(): end
  }
}
```

Fig. 3.4: Protocolo con múltiples destinos a través de enumeraciones

3.2. Aliasing

Una de las situaciones problemáticas a la hora de analizar código estáticamente es cuando hay varias referencias vivas del mismo objeto, también conocido como **Aliasing**. En particular, dificulta (y a veces imposibilita) asegurarse que una referencia en otra parte del código no haya modificado el estado del objeto, invalidando lo que el analizador asume como correcto, y potencialmente estropeando el resultado del mismo.

Para solventarlo, *JaTyC* impone una restricción dura sobre el aliasing, obligando a los programadores a usar objetos protocolados **linealmente**. Esto significa que, para un objeto protocolado, una única referencia va a ser capaz de modificar su typestate, llamada *referencia lineal*. Esto genera una limitación en el poder de expresión para los desarrolladores, pero permite a la herramienta verificar que los objetos sigan y completen su protocolo. Las demás referencias, denominadas *referencias compartidas*, tendrán acceso solamente a sus métodos *anytime*.

Los **Anytime Methods**, o "métodos de cualquier momento", son todos los métodos definidos por una clase que no están mencionados por su protocolo, o todos los métodos de una clase en caso de no tener protocolo. Estos pueden ser llamados para una objeto en cualquier punto de la ejecución del programa, sin importar el estado en el que el mismo se encuentre. Dado que estos métodos se asumen no relevantes para el protocolo, y para preservar la linealidad, los mismos no pueden modificar los valores de los colaboradores internos de la instancia, ni hacerlos transicionar a otro typestate.

Las variables pierden la referencia lineal a su objeto, de tenerla, cuando se asigna a otra variable, parámetro, o colaborador interno. Es decir, asignar una referencia lineal a una variable nueva "transmite la linealidad" de la vieja a la nueva, mientras que la vieja pasa a ser compartida.

En la Figura 3.5 puede observarse un ejemplo de código cliente utilizando un objeto de tipo `Stack` que está protocolado. El mismo tiene un protocolo muy simple, similar al de la Figura 2.3, el cual incluye al método `push()` (en otras palabras, no es un método *anytime*). El código cliente comienza definiendo una variable `s1`, y asignándole una nueva instancia de `Stack`, transformándola en una referencia lineal. Acto seguido, se define una nueva variable `s2` y se la hace apuntar al mismo objeto. Esto transfiere la linealidad de `s1` a `s2`, y `s1` pasa a ser una referencia compartida, sin acceso a los métodos protocolados. Notemos que al correr la herramienta, la misma reporta como un error la llamada al método `push()` desde la referencia compartida `s1` y no la lineal `s2`.

```
public class Main {
  public static void main(String args[]) throws Exception {
    Stack s1 = new Stack(5);
    Stack s2 = s1;
    s2.push(5);
    s1.push(5);
  }
}
```

```
Main.java:6: error: Cannot call [push] on Shared{Stack}
    s1.push(5);
```

Fig. 3.5: JaTyC reportando un error al llamar métodos protocolados en referencia compartida

3.3. Anotaciones Adicionales

Además de la introducción de los protocolos, tanto con la definición en los archivos *.protocol* como con la anotación `@Typestate` para suscribirse, *JaTyC* provee otras anotaciones útiles para definir comportamiento esperado en los programas.

3.3.1. Nullable

Por defecto, *JaTyC* considera a todas las variables de un programa como no anulables, a diferencia del sistema de tipos por defecto de Java. Para definir que la variable puede tomar el valor de `null`, se utiliza la notación `@nullable`, como se muestra en la Figura 3.6. De no utilizar la anotación y asignar `null` a una variable no anulable, la herramienta reportará un error. A través de este mecanismo, el checker previene errores típicos en código de Java tales como los de *Null Pointer*.

Esta anotación puede utilizarse en la definición de variables y colaboradores internos, argumentos de funciones y valores de retorno.

```
public @Nullable int next() {
  return this.items[this.index++];
}
```

Fig. 3.6: Uso de anotación `@Nullable` para el método `next()` de un iterador

3.3.2. Requires

La anotación `@Requires` puede ser utilizada en los parámetros de un método para indicar en qué typestate (o conjunto de typestates) se espera que el objeto provisto se encuentre. Un ejemplo de esto puede observarse en la Figura 3.7, donde se muestran un par de métodos para un controlador de un robot. En ellos, requieren que el robot pasado por parámetro esté en determinados estados obligatoriamente.

De no proveer esta anotación, *JaTyC* asume que el objeto recibido por el método es una referencia compartida, y permitirá solamente utilizar sus métodos *Anytime*. A su vez, de sí proveerla, se debe de tener en cuenta la finalización de protocolo del objeto. Es decir, cuando se salga del scope del método se perderá la única referencia lineal del mismo, y por

ende su protocolo debe de completarse antes (a no ser de utilizar la anotación `@Ensures`, de la que hablaremos en la próxima sección).

```
public void moveRight(@Requires("TopLeft") Robot robot) {
  robot.moveRight();
}

private void moveLeft(@Requires({"TopRight", "BotRight"}) Robot robot) {
  robot.moveLeft();
}
```

Fig. 3.7: Uso de anotación `@Requires` para un solo typestate y para un conjunto de typestates

3.3.3. Ensures

Por último, la herramienta provee la anotación `@Ensures`, utilizada cuando se quiere indicar el typestate (o conjunto de typestates) en el que se encuentra el valor de retorno de un método. Un ejemplo puede observarse en la Figura 3.8, donde se ve un método *getter* de un controlador de robots. En él, se define que el valor devuelto será una referencia lineal a un robot que se encuentra exactamente en el estado `TopLeft`. Al retornar, el código cliente podrá acceder a los métodos protocolados del robot.

De no utilizar esta anotación, se asume que la referencia devuelta será compartida, y solamente sus métodos *Anytime* estarán disponibles. A su vez, de no utilizarla y devolver un objeto que tiene protocolo (tal como el Robot de la Figura 3.8), se requiere que el mismo lo finalice antes de retornar del método (ya sea a través de una transición a `end` o un estado descartable). De no ser así, se perdería la única referencia lineal al salir del scope, y nunca llegaría el objeto a completar su protocolo. Por ejemplo, de retornar un lector de archivos, se espera que el método finalice luego de haber cerrado el archivo.

```
public @Ensures("TopLeft") Robot getRobot() {
  return this.robot;
}
```

Fig. 3.8: Uso de anotación `@Ensures` en el valor de retorno de un método

```
public static void pushToStack(
  @Requires("Init") @Ensures("Init") final Stack stack,
  int e
) {
  if (!stack.isFull()) {
    stack.push(e);
  }
}
```

Fig. 3.9: Uso de anotación `@Ensures` en un parámetro de entrada de un método

Durante nuestra experimentación, notamos que la anotación también puede ser utilizada en los parámetros de entrada de un método. Esto no es mencionado por la documentación oficial de la herramienta, con lo cual no podemos afirmar que es comportamiento

esperado. De todos modos, sentimos que es relevante mencionarlo dado que permite más flexibilidad a la hora de utilizar los objetos protocolados. Esto es porque, de combinarlo con la anotación `@Requires` en un parámetro de entrada, es posible devolver la referencia lineal del objeto provisto por el cliente, y no una compartida. De esta forma, se consigue evitar tener que obligatoriamente completar el protocolo del parámetro dentro del scope del método.

Un ejemplo de uso de ambas anotaciones puede observarse en la Figura 3.9, donde se opera con un `Stack`. El método declara que el stack provisto deberá encontrarse en el typestate `Init`, y que lo devolverá en el mismo estado. Para que funcione, *JaTyC* requiere que el parámetro sea `final`, es decir, que el objeto al que referencia no cambie durante la ejecución del método.

3.4. Subtipado

Una de las características más interesantes de la herramienta, similar a lo mencionado en el capítulo 2.1.2, es el soporte para **subtipado**. Es una de las funcionalidades distintivas de *JaTyC* comparada a su predecesora Mungo, y crucial dentro del paradigma de los objetos.

De tener una clase protocolada extendiendo a otra también protocolada, *JaTyC* se asegura que el protocolo del subtipo incluya al del supertipo. En otras palabras, toda secuencia válida de llamadas de métodos para el tipo padre deben de seguir siéndolo para el tipo hijo. El subtipo tiene solamente la capacidad de **agregar** nuevas secuencias de llamadas de métodos. En cierto sentido, modelando el protocolo de un objeto como una máquina de estados finita, el autómata del subtipo debe preservar el comportamiento del autómata del supertipo.

```
1 typestate Iterator {
2   HasNext = {
3     boolean hasNext(): <true: Next,
      false: end>
4   }
5   Next = {
6     boolean hasNext(): <true: Next,
      false: end>,
7     Object next(): HasNext
8   }
9 }
```

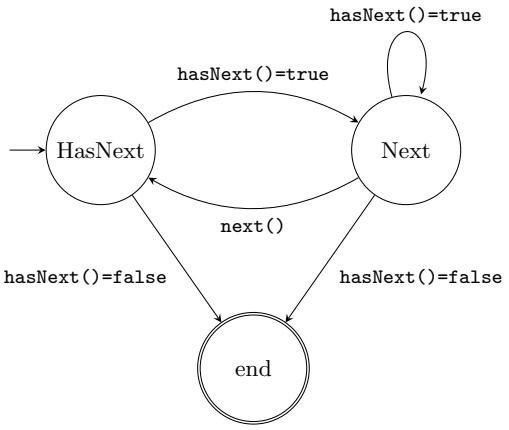


Fig. 3.10: Protocolo `Iterator` y su autómata

Esto es así dado que se pretende que el subtipo se "comporte como" el supertipo. En las Figuras 3.10 y 3.11 se pueden ver los protocolos para una interfaz `Iterator` y una clase `LoopIterator` que la implementa. La clase funciona de tal manera que, como indica su nombre, al llegar al final de la colección reinicia su puntero al inicio de la misma, eliminando la necesidad de consultar si hay elemento siguiente (`hasNext()`). Notemos que todas las secuencias de métodos válidas para `Iterator` son llamadas infinitas a `hasNext()` y una única llamada a `next()` cada vez que haya un elemento siguiente. Observemos que

```
typestate LoopIterator {
  Init = {
    boolean hasNext(): <true: Init,
    false: Init>,
    Object next(): Init,
    drop: end
  }
}
```

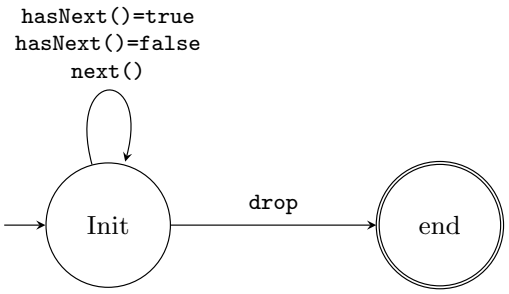
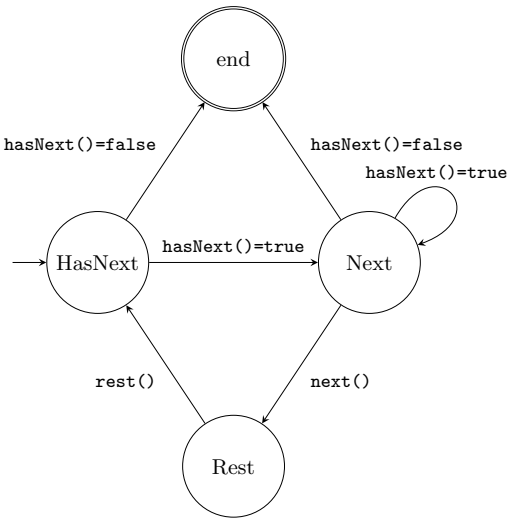


Fig. 3.11: Protocolo `LoopIterator` y su autómata

el `LoopIterator` permite todo eso y más, dado que ambos métodos pueden llamarse sin restricciones. De tener un objeto de tipo `Iterator`, uno espera que se comporte como uno, y que por ende siga su protocolo. Si el de `LoopIterator` no permitiese *al menos* todas las secuencias de llamadas de métodos de su interfaz, cualquier código cliente puede levantar una excepción al utilizar un objeto `Iterator`. Esto es porque, incluso siguiendo el contrato de uso de la interfaz, se podría estar rompiendo con el de la implementación.

```
typestate TiredIterator {
  HasNext = {
    boolean hasNext(): <true: Next,
    false: end>
  }
  Next = {
    boolean hasNext(): <true: Next,
    false: end>,
    Object next(): Rest
  }
  Rest = {
    void rest(): HasNext
  }
}
```




```
TiredIterator.java:6: error: [hasNext] transition(s) in [HasNext] of
    Iterator.protocol are not included in [Rest] of TiredIterator.protocol
public class TiredIterator implements Iterator {
```

Fig. 3.12: Protocolo `TiredIterator`, su autómata y el error que reporta el checker

Un ejemplo de *JaTyC* previniendo esta clase de situaciones puede observarse en la Figura 3.11. La clase *TiredIterator* implementa la misma interfaz, pero su protocolo agrega un nuevo estado *Rest* incluyendo al método `rest()`. Analizando el comportamiento del protocolo, luego de cada llamada a `next()`, necesariamente se deberá de "dejar al iterador descansar", llamando a su nuevo método. Dada esta nueva implementación, se puede fácilmente ver que no permite las mismas secuencias de llamadas de métodos que su interfaz. Por ejemplo, la secuencia `hasNext()`, `next()` y luego `hasNext()`, permitida por `Iterator`, no se puede realizar siguiendo el protocolo de `TiredIterator`, ya que necesariamente se debe de llamar a `rest()`. En la misma figura puede también observarse el error reportado

por la herramienta al analizar los protocolos.

Por otro lado, *JaTyC* también permite que una clase no protocolada extienda a otra con protocolo. En este caso, el subtipo sigue el protocolo del supertipo, y todos sus nuevos métodos son considerados *anytime*. Finalmente, se puede tener una clase protocolada extendiendo a una sin protocolo. En esta situación, se asumen todos los métodos de la clase padre como *anytime*, y la clase hijo funciona como una protocolada regular.

Como parte de esta funcionalidad, *JaTyC* también introduce lógica para manejar correctamente la conversión de tipos, o *casting*, tanto implícitos como explícitos.

3.5. Impacto

Con todas las características mencionadas anteriormente, *JaTyC* consigue determinar en tiempo de compilación que todos los objetos sigan y completen sus protocolos, de tenerlos. En otras palabras, verifica que los métodos de los objetos sean llamados en orden correcto y que los objetos sean utilizados hasta el final. A su vez, con sus anotaciones permite detectar potenciales errores de punteros nulos, y especificar en métodos los typestates esperables para parámetros y valores de retorno. Finalmente, da soporte al subtipado, permitiendo a los usuarios hacer herencia entre cualquier clase y manteniendo la correcta verificación de protocolos.

3.6. Implementación

3.6.1. Algoritmo

3.6.2. Checker Framework

Bibliografía

[1] Jonathan Aldrich, Joshua Sunshine, Darpan Saini, and Zachary Sparks. Typestate-oriented programming. In *Proceedings of the 24th ACM SIGPLAN Conference Companion on Object Oriented Programming Systems Languages and Applications*, OOPSLA '09, page 1015–1022, New York, NY, USA, 2009. Association for Computing Machinery.

[2] Amazon Web Services Labs. Crypto Policy Compliance Checker. `https://github.com/awslabs/aws-crypto-policy-compliance-checker`. Accessed: 2026-09-07.

[3] Amazon Web Services Labs. KMS Compliance Checker. `https://github.com/awslabs/aws-kms-compliance-checker`. Accessed: 2026-09-07.

[4] Nels E. Beckman, Duri Kim, and Jonathan Aldrich. An empirical study of object protocols in the wild. In *Proceedings of the 25th European Conference on Object-Oriented Programming*, ECOOP'11, page 2–26, Berlin, Heidelberg, 2011. Springer-Verlag.

[5] Nels E. Beckman, Duri Kim, and Jonathan Aldrich. An empirical study of object protocols in the wild. In *Proceedings of the 25th European Conference on Object-Oriented Programming*, ECOOP'11, page 2–26, Berlin, Heidelberg, 2011. Springer-Verlag.

[6] Kevin Bierhoff and Jonathan Aldrich. Lightweight object specification with typestates. *SIGSOFT Softw. Eng. Notes*, 30(5):217–226, September 2005.

[7] Kevin Bierhoff and Jonathan Aldrich. Modular typestate checking of aliased objects. *SIGPLAN Not.*, 42(10):301–320, October 2007.

[8] Checker Framewor. The checker framework. `https://checkerframework.org`. Accessed: 2026-09-07.

[9] Ronald Garcia, Éric Tanter, Roger Wolff, and Jonathan Aldrich. Foundations of typestate-oriented programming. *ACM Trans. Program. Lang. Syst.*, 36(4), October 2014.

[10] JastAdd. JastAdd. `https://jastadd.cs.lth.se/web/`. Accessed: 2026-09-29.

[11] JastAdd. JastAdd - Applications. `https://jastadd.cs.lth.se/web/applications.php`. Accessed: 2026-09-29.

[12] JetBrains. Kotlin language. `https://kotlinlang.org`. Accessed: 2026-09-29.

[13] Dimitrios Kouzapas, Ornela Dardha, Roly Perera, and Simon J. Gay. Typechecking protocols with mungo and stmungo. In *Proceedings of the 18th International Symposium on Principles and Practice of Declarative Programming*, PPDP '16, page 146–159, New York, NY, USA, 2016. Association for Computing Machinery.

[14] João Mota. Coping with the reality: adding crucial features to a typestate-oriented language. `https://github.com/jdmota/java-typestate-checker/blob/master/docs/msc-thesis.pdf`. Accessed: 2026-10-01.

[15] João Mota and Lorenzo Bacchiani. Jatyc - mungo comparison. `https://github.com/jdmota/java-typestate-checker/wiki/Mungo-comparison`. Accessed: 2026-09-07.

[16] João Mota and Lorenzo Bacchiani. Java typestate checker tool. `https://github.com/jdmota/java-typestate-checker`. Accessed: 2026-09-05.

[17] João Mota and Lorenzo Bacchiani. Java typestate checker tool - documentation. `https://github.com/jdmota/java-typestate-checker/wiki/Documentation`. Accessed: 2026-09-29.

[18] Oracle. Interface Iterator. `https://docs.oracle.com/javase/8/docs/api/java/util/Iterator.html`. Accessed: 2026-09-05.

[19] R E Strom and S Yemini. Typestate: A programming language concept for enhancing software reliability. *IEEE Trans. Softw. Eng.*, 12(1):157–171, January 1986.

[20] Carnegie Mellon University. Plaid programming language. `https://www.cs.cmu.edu/~aldrich/plaid/`. Accessed: 2026-09-29.

[21] University of Glasgow. Mungo. `https://www.dcs.gla.ac.uk/research/mungo/`. Accessed: 2026-09-07.

[22] University of Glasgow. Mungo - Oficial Repository. `https://bitbucket.org/abcd-glasgow/mungo-tools/src/master/`. Accessed: 2026-10-01.